

LAPORAN PRAKTIKUM PEMROGRAMAN BACKEND LANJUT

MODUL 6: AUTHORIZATION & ROLE BASED ACCESS CONTROL (RBAC)

Mata Kuliah	: Praktikum Pemrograman Backend Lanjut (2 SKS) — Semester 5
Program Studi	: D4 Teknik Informatika — Fakultas Vokasi, Universitas Airlangga
Topik Pembahasan	: Role Based Access Control (RBAC), Middleware, & Ownership (ABAC)
Repository URL	: https://github.com/odaynih2307-hue/Tugas-Go.git
Status Verifikasi	: 100% LULUS (Langkah 1–9, Tugas Mandiri C.1–C.4 & Pengujian Negatif)

Ringkasan Praktikum:

Pada Modul 6 ini, sistem keamanan API diperluas secara komprehensif dari autentikasi murni (Modul 5) menjadi otorisasi berbasis peran (**Role Based Access Control - RBAC**) dan kontrol akses berbasis kepemilikan (**Ownership ABAC**). Seluruh hak akses disimpan secara relasional di PostgreSQL dan dimuat sekali saat inisialisasi aplikasi ke dalam memori (PermissionSet) dengan prinsip *Fail Closed*. Entitas users dan students kini terlindungi secara kokoh dari serangan *Broken Access Control* (OWASP Top 10 A01), *Insecure Direct Object Reference (IDOR)*, dan *Timing Attacks*. Seluruh matriks hak akses dan pengujian negatif telah diverifikasi dengan bukti status code aktual.

1. Landasan Teori & Keputusan Arsitektural

1.1 Authentication vs Authorization:

Authentication menjawab 'Siapa Anda?' (memverifikasi kredensial username/password dan menghasilkan token JWT).

Authorization menjawab 'Anda berhak melakukan apa?' (mengevaluasi role dan permission yang melekat pada identitas pemanggil). Dalam rantai middleware, RequireAuth harus selalu dijalankan sebelum RequirePermission agar identitas tersedia di context sebelum hak akses diperiksa.

1.2 Pemisahan Evaluasi Akses: Middleware vs Service:

Keputusan akses yang independen terhadap isi data (misal: melihat daftar user/student, membuat student, menghapus student) diletakkan di Middleware (RequirePermission) agar file route terbaca sebagai peta hak akses. Keputusan akses yang bergantung pada isi data (misal: mengubah profil atau mahasiswa milik sendiri) diletakkan di layer Service (CanAccessStudent) untuk menghindari polusi query database di lapisan middleware.

1.3 Prinsip Fail Closed, Timing Attack Mitigation, & Stateless JWT:

- **Fail Closed:** Saat terjadi keraguan (role/permission tidak dikenal, pointer nil), sistem secara mutlak menolak akses (403/401).
- **Mitigasi Timing Attack:** Pemeriksaan hak akses dilakukan sebelum query FindByID agar waktu respon seragam dan mencegah ID enumeration.
- **Stateless JWT:** Perubahan role di DB tidak seketika mengubah hak pada token aktif lama; access token diberi umur pendek (15 menit).

2. Potongan Kode Kunci & Implementasi

2.1 Skema Relasional RBAC & Student (`migrations/003\_rbac.sql` & `004\_student\_permissions.sql`)

```
CREATE TABLE IF NOT EXISTS role_permissions (
  role_name VARCHAR(20) NOT NULL REFERENCES roles(name) ON DELETE CASCADE,
  permission_name VARCHAR(50) NOT NULL REFERENCES permissions(name) ON DELETE CASCADE,
  PRIMARY KEY (role_name, permission_name)
);
-- Tambah kolom owner_id pada students yang terikat foreign key ke users
ALTER TABLE students ADD COLUMN IF NOT EXISTS owner_id INTEGER;
ALTER TABLE students ADD CONSTRAINT fk_students_owner FOREIGN KEY (owner_id) REFERENCES users(id);
```

2.2 In-Memory PermissionSet (`helper/authz.go`)

```
type PermissionSet struct { byRole map[string]map[string]struct{} }
func (p *PermissionSet) Can(role, permission string) bool {
  if p == nil { return false }
  perms, ok := p.byRole[role]
  if !ok { return false }
  _, granted := perms[permission]
  return granted
}
```

2.3 Middleware RequirePermission (`middleware/authz.go`)

```
func RequirePermission(perms *helper.PermissionSet, permission string) fiber.Handler {
  return func(c *fiber.Ctx) error {
    user, ok := helper.CurrentUser(c)
    if !ok { return helper.Fail(c, fiber.StatusUnauthorized, "belum terautentikasi") }
    if !perms.Can(user.Role, permission) {
      return helper.Fail(c, fiber.StatusForbidden, "role "+user.Role+" tidak memiliki hak "+permission)
    }
    return c.Next()
  }
}
```

2.4 Aturan Kepemilikan Mahasiswa Murni (`app/service/student\_authz\_rules.go`)

```
func CanAccessStudent(current model.AuthUser, ownerID int, perms *helper.PermissionSet, anyPerm string) bool {
  if ownerID > 0 && current.UserID == ownerID { return true } // Kepemilikan selalu boleh
  return perms.Can(current.Role, anyPerm) // Cek permission global
}
```

2.5 Proteksi Create Student & Invariant (`app/service/student\_service.go`)

```
// owner_id dikunci otomatis dari token JWT pemanggil, tidak bisa dipalsukan lewat body JSON
baru, err := s.repo.Create(ctx, model.Student{
  NIM: req.NIM, Name: req.Name, Grade: req.Grade, IsActive: isActive, OwnerID: current.UserID,
})
```

3. Tabel Matriks Hak Akses & Hasil Pengujian

3.1 Matriks Hak Akses Endpoint Users (Langkah 9)

Endpoint & Skenario	Method	admin	staff	user	NoAuth	Status
GET /users (Daftar semua user)	GET	200	200	403	401	PASS
GET /users/:id (Diri Sendiri)	GET	200	200	200	401	PASS
GET /users/:id (Orang Lain)	GET	200	200	403	401	PASS
PUT /users/:id (Orang Lain)	PUT	200	403	403	401	PASS
DELETE /users/:id (Orang Lain)	DELETE	204	403	403	401	PASS
DELETE /users/:id (Akun Sendiri)	DELETE	403	403	403	401	PASS
PATCH /users/:id/role (Orang Lain)	PATCH	200	403	403	401	PASS
PATCH /users/:id/role (Diri Sendiri)	PATCH	422	403	403	401	PASS

3.2 Matriks Hak Akses Endpoint Students (Tugas Mandiri C.3)

Endpoint & Skenario	Method	admin	staff	user (Own)	user (Other)	NoAuth	Status
GET /students	GET	200	200	403	403	401	PASS
POST /students	POST	201	201	403	403	401	PASS
GET /students/:id (Miliknya)	GET	200	200	200	403	401	PASS
GET /students/:id (Orang Lain)	GET	200	200	—	403	401	PASS
PUT /students/:id (Miliknya)	PUT	200	403	200	403	401	PASS
PUT /students/:id (Orang Lain)	PUT	200	403	—	403	401	PASS
PATCH /students/:id (Miliknya)	PATCH	200	403	200	403	401	PASS
PATCH /students/:id (Orang Lain)	PATCH	200	403	—	403	401	PASS
DELETE /students/:id	DELETE	204	403	403	403	401	PASS

4. Bukti Pengujian Negatif & Analisis Keamanan (C.3)

1. Percobaan Pemalsuan owner_id saat POST /students:

- Perintah: `curl -X POST /api/v1/students -H 'Authorization: Bearer $TOKEN_STAFF' -d '{"nim":"23081010099","owner_id":1}'`
- Hasil: **201 Created** dengan nilai respons "owner_id": 5 (sesuai ID akun staff pemanggil; parameter owner_id diabaikan).

2. Percobaan IDOR (User Biasa Mengubah Mahasiswa Milik Orang Lain):

- Perintah: `curl -X PUT /api/v1/students/7 -H 'Authorization: Bearer $TOKEN_USER_OWNER' -d '{...}'`
- Hasil: **403 Forbidden** ({ "message": "tidak berhak mengubah data student ini" }).

3. Percobaan Admin Menurunkan Role Dirinya Sendiri:

- Perintah: `curl -X PATCH /api/v1/users/4/role -H 'Authorization: Bearer $TOKEN_ADMIN' -d '{"role":"user"}'`
- Hasil: **422 Unprocessable Entity** ({ "errors": { "role": "tidak boleh mengubah role diri sendiri" } }).

4. Pembuktian Stateless JWT (Token Lama vs Token Baru):

- Ketika role akun dinaikkan dari 'user' menjadi 'staff' oleh admin, pemanggilan GET /students menggunakan token lama menghasilkan **403 Forbidden**. Setelah login ulang dan memperoleh token baru, request menghasilkan **200 OK**.

5. Jawaban Analisis Singkat (C.4) & Eksplorasi Mandiri

1. Mengapa pemeriksaan kepemilikan tidak dapat dipindahkan ke middleware?

Pemeriksaan kepemilikan seperti `CanAccessStudent(current, student.OwnerID, s.perms, "student:read:any")` tidak dapat dipindahkan ke middleware karena middleware hanya memiliki konteks parameter URL dan claims token JWT, namun belum mengetahui isi baris data spesifik di database (khususnya kolom owner_id). Jika middleware dipaksa melakukan query ke database, maka lapisan middleware akan tercemar oleh dependensi repository/basis data yang melanggar prinsip *Clean Architecture*. Oleh karena itu, pemeriksaan kepemilikan secara tepat berada di layer Service setelah data dimuat.

2. Risiko pemisahan pemeriksaan akses pada route vs service dan mitigasinya:

Risiko konkretnya adalah kelalaian pengembang (*developer omission / missed enforcement*) di mana saat membuat endpoint baru, programmer berasumsi otorisasi sudah dihandle di middleware sehingga lupa memasang validasi kepemilikan di service, atau sebaliknya. Kami memitigasi risiko ini melalui: (1) Konvensi penamaan permission yang terstruktur (domain:action:scope) dan dokumentasi eksplisit pada file route, (2) Isolasi aturan otorisasi ke dalam fungsi murni (`student_authz_rules.go`) yang diverifikasi unit test, serta (3) Suite pengujian matriks otomatis end-to-end untuk setiap kombinasi role.

3. Kebutuhan 'Dosen wali hanya boleh melihat mahasiswa bimbingannya' — Apakah RBAC memadai?

RBAC murni tidak lagi memadai karena keputusan akses tidak semata-mata bergantung pada nama peran, melainkan relasi dinamis antara atribut subjek (ID Dosen) dan atribut objek (kolom advisor_id pada Mahasiswa). Sistem perlu dielevasi menjadi **ABAC / ReBAC**. Yang perlu ditambahkan: (1) Kolom advisor_id INTEGER REFERENCES users(id) pada tabel students, dan (2) Aturan di layer service: `if current.Role == "advisor" && student.AdvisorID == current.UserID { return true }`.

Jawaban Eksplorasi Mandiri:

- **Urutan Terbalik (RequirePermission sebelum RequireAuth):** Menghasilkan **401 Unauthorized** karena claims belum diekstrak (fail closed).
- **Hapus Role-Permission Tanpa Restart:** Hak akses belum berubah karena pemetaan dimuat sekali saat startup ke in-memory PermissionSet.
- **Menambah Role 'auditor':** Memerlukan **0 baris perubahan kode Go** (cukup 2 baris query SQL di database lalu restart aplikasi).
- **Salah Ketik Nama Permission ('user:lis'):** Menghasilkan **403 Forbidden** karena fail closed, mencegah celah keamanan.
- **Memindahkan Cek Otorisasi Setelah FindByID:** Menghilangkan proteksi *Timing Attack* (memungkinkan ID enumeration).

6. Kesimpulan

Sistem otorisasi RBAC dan Ownership Access Control telah berhasil dirancang, diimplementasikan, dan diuji secara komprehensif. Seluruh persyaratan Modul 6 (Langkah Panduan 1–9, Tugas Mandiri C.1–C.4, mitigasi celah OWASP Broken Access Control, dan timing attack) telah dipenuhi dengan kepatuhan penuh terhadap arsitektur bersih (*Clean Architecture*).